

1. Student Registration Form with SQLite (CRUD Operations).

```
from tkinter import *
import sqlite3
import tkinter.ttk as ttk
import tkinter.messagebox as tkMessageBox

def Database():
    conn = sqlite3.connect("computer.db")
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS member (
            mem_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            city TEXT NOT NULL,
            rollno INTEGER UNIQUE NOT NULL
        )
    """)

    conn.commit()
    return conn, cursor

def ValidateFields():
    if NAME.get().strip() == "":
        txt_result.config(text="Please enter name!", fg="red")
        return False

    if CITY.get().strip() == "":
        txt_result.config(text="Please enter city!", fg="red")
        return False

    if ROLLNO.get().strip() == "":
        txt_result.config(text="Please enter roll number!", fg="red")
        return False

    if not ROLLNO.get().strip().isdigit():
        txt_result.config(text="Roll number must be a number!", fg="red")
        return False

    return True

def Create():
    if not ValidateFields():
        return

    try:
        conn, cursor = Database()

        cursor.execute(
            "INSERT INTO member (name, city, rollno) VALUES (?, ?, ?)",
            (
```

```
        NAME.get().strip(),
        CITY.get().strip(),
        int(ROLLNO.get().strip())
    )
)

conn.commit()
cursor.close()
conn.close()

ClearFields()
Read()

txt_result.config(
    text="Student data created successfully!",
    fg="green"
)

except sqlite3.IntegrityError:
    txt_result.config(
        text="Roll number already exists!",
        fg="red"
    )

except sqlite3.Error as e:
    txt_result.config(
        text="Database error: " + str(e),
        fg="red"
    )

def Read():
    tree.delete(*tree.get_children())

    try:
        conn, cursor = Database()

        cursor.execute(
            "SELECT * FROM member ORDER BY city ASC"
        )

        records = cursor.fetchall()

        for data in records:
            tree.insert(
                "",
                END,
                values=(data[1], data[2], data[3])
            )

        cursor.close()
        conn.close()
```

```
txt_result.config(
    text="Successfully read data from database",
    fg="black"
)

except sqlite3.Error as e:
    txt_result.config(
        text="Database error: " + str(e),
        fg="red"
    )

def Update():
    if not ValidateFields():
        return

    try:
        conn, cursor = Database()

        cursor.execute(
            """
            UPDATE member
            SET name = ?, city = ?
            WHERE rollno = ?
            """,
            (
                NAME.get().strip(),
                CITY.get().strip(),
                int(ROLLNO.get().strip())
            )
        )

        if cursor.rowcount == 0:
            txt_result.config(
                text="No student found with this roll number!",
                fg="red"
            )
        else:
            conn.commit()

            txt_result.config(
                text="Student data updated successfully!",
                fg="green"
            )

        cursor.close()
        conn.close()

        ClearFields()
        Read()

    except sqlite3.Error as e:
        txt_result.config(
```

```
        text="Database error: " + str(e),
        fg="red"
    )

def Delete():
    if ROLLNO.get().strip() == "":
        txt_result.config(
            text="Please enter roll number!",
            fg="red"
        )
        return

    if not ROLLNO.get().strip().isdigit():
        txt_result.config(
            text="Roll number must be a number!",
            fg="red"
        )
        return

    try:
        result = tkMessageBox.askyesno(
            "Delete Student",
            "Are you sure you want to delete this student?"
        )

        if not result:
            return

        conn, cursor = Database()

        cursor.execute(
            "DELETE FROM member WHERE rollno = ?",
            (int(ROLLNO.get().strip()),)
        )

        if cursor.rowcount == 0:
            txt_result.config(
                text="No student found with this roll number!",
                fg="red"
            )
        else:
            conn.commit()

            txt_result.config(
                text="Student deleted successfully!",
                fg="green"
            )

        cursor.close()
        conn.close()

        ClearFields()
```

```
Read()

except sqlite3.Error as e:
    txt_result.config(
        text="Database error: " + str(e),
        fg="red"
    )

def ClearFields():
    NAME.set("")
    CITY.set("")
    ROLLNO.set("")

def SelectItem(event):
    selected = tree.focus()

    if not selected:
        return

    values = tree.item(selected, "values")

    if values:
        NAME.set(values[0])
        CITY.set(values[1])
        ROLLNO.set(values[2])

def Exit():
    result = tkMessageBox.askyesno(
        "Student Registration Form",
        "Are you sure you want to exit?",
        icon="warning"
    )

    if result:
        root.destroy()

root = Tk()
root.title("Student Registration Form")

screen_width = root.winfo_screenwidth()
screen_height = root.winfo_screenheight()

width = 900
height = 500

x = int((screen_width / 2) - (width / 2))
y = int((screen_height / 2) - (height / 2))

root.geometry(f"{width}x{height}+{x}+{y}")
root.resizable(False, False)

NAME = StringVar()
```

```
CITY = StringVar()
ROLLNO = StringVar()

Top = Frame(
    root,
    width=900,
    height=70,
    bd=8,
    relief="raise"
)
Top.pack(side=TOP, fill=X)

Left = Frame(
    root,
    width=300,
    height=430,
    bd=8,
    relief="raise"
)
Left.pack(side=LEFT, fill=Y)

Right = Frame(
    root,
    width=600,
    height=430,
    bd=8,
    relief="raise"
)
Right.pack(side=RIGHT, fill=BOTH, expand=True)

txt_title = Label(
    Top,
    text="Student Registration Form",
    font=("Arial", 24, "bold")
)
txt_title.pack(pady=5)

Forms = Frame(
    Left,
    width=280,
    height=250
)
Forms.pack(side=TOP, pady=20)

txt_name = Label(
    Forms,
    text="NAME:",
    font=("Arial", 14),
    bd=10
)
txt_name.grid(row=0, column=0, sticky="e")
```

```
txt_city = Label(
    Forms,
    text="CITY:",
    font=("Arial", 14),
    bd=10
)
txt_city.grid(row=1, column=0, sticky="e")

txt_rollno = Label(
    Forms,
    text="ROLL NO:",
    font=("Arial", 14),
    bd=10
)
txt_rollno.grid(row=2, column=0, sticky="e")

name = Entry(
    Forms,
    textvariable=NAME,
    width=25,
    font=("Arial", 12)
)
name.grid(row=0, column=1)

city = Entry(
    Forms,
    textvariable=CITY,
    width=25,
    font=("Arial", 12)
)
city.grid(row=1, column=1)

rollno = Entry(
    Forms,
    textvariable=ROLLNO,
    width=25,
    font=("Arial", 12)
)
rollno.grid(row=2, column=1)

Buttons = Frame(
    Left,
    width=280,
    height=150,
    bd=8,
    relief="raise"
)
Buttons.pack(side=BOTTOM, fill=X)

txt_result = Label(
    Buttons,
    text="",
```

```
        font=("Arial", 10)
    )
    txt_result.pack(side=TOP, pady=5)

    btn_create = Button(
        Buttons,
        width=8,
        text="Create",
        command=Create
    )
    btn_create.pack(side=LEFT, padx=2)

    btn_read = Button(
        Buttons,
        width=8,
        text="Read",
        command=Read
    )
    btn_read.pack(side=LEFT, padx=2)

    btn_update = Button(
        Buttons,
        width=8,
        text="Update",
        command=Update
    )
    btn_update.pack(side=LEFT, padx=2)

    btn_delete = Button(
        Buttons,
        width=8,
        text="Delete",
        command=Delete
    )
    btn_delete.pack(side=LEFT, padx=2)

    btn_exit = Button(
        Buttons,
        width=8,
        text="Exit",
        command=Exit
    )
    btn_exit.pack(side=LEFT, padx=2)

    scrollbar = Scrollbar(
        Right,
        orient=VERTICAL
    )
    scrollbar.pack(
        side=RIGHT,
        fill=Y
    )
```

```
scrollbarx = Scrollbar(  
    Right,  
    orient=HORIZONTAL  
)  
scrollbarx.pack(  
    side=BOTTOM,  
    fill=X  
)  
  
tree = ttk.Treeview(  
    Right,  
    columns=("Name", "City", "Rollno"),  
    show="headings",  
    selectmode="browse",  
    yscrollcommand=scrollbary.set,  
    xscrollcommand=scrollbarx.set  
)  
  
scrollbary.config(  
    command=tree.yview  
)  
  
scrollbarx.config(  
    command=tree.xview  
)  
  
tree.heading(  
    "Name",  
    text="Name",  
    anchor=W  
)  
  
tree.heading(  
    "City",  
    text="City",  
    anchor=W  
)  
  
tree.heading(  
    "Rollno",  
    text="Roll No",  
    anchor=W  
)  
  
tree.column(  
    "Name",  
    width=180,  
    anchor=W  
)  
  
tree.column(  

```

```
"City",
width=150,
anchor=W
)

tree.column(
    "Rollno",
    width=100,
    anchor=W
)

tree.pack(
    fill=BOTH,
    expand=True,
    padx=5,
    pady=5
)

tree.bind(
    "<ButtonRelease-1>",
    SelectItem
)

Database()[0].close()

Read()

root.mainloop()
```

Output:

Name	City	Roll No
Vansh	Valsad	39

2. Student Registration with SQLite (College Database).

```
from tkinter import *
import sqlite3
from tkinter import ttk, messagebox

def Database():
    conn = sqlite3.connect("college.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS student (
            stud_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            city TEXT,
            rollno INTEGER
        )
    """)
    conn.commit()
    conn.close()

def Create():
    if NAME.get()==" " or CITY.get()==" " or ROLLNO.get()==" ":
        lbl_msg.config(text="Please fill all fields!", fg="red")
    else:
        conn = sqlite3.connect("college.db")
        cursor = conn.cursor()
        cursor.execute("INSERT INTO student (name, city, rollno) VALUES (?, ?, ?)",
            (NAME.get(), CITY.get(), ROLLNO.get()))
        conn.commit()
        conn.close()
        NAME.set(""); CITY.set(""); ROLLNO.set("")
        lbl_msg.config(text="Data Inserted!", fg="green")
        Read()

def Read():
    tree.delete(*tree.get_children())
    conn = sqlite3.connect("college.db")
    cursor = conn.cursor()
    cursor.execute("SELECT name, city, rollno FROM student")
    for row in cursor.fetchall():
        tree.insert("", END, values=row)
    conn.close()

def Update():
    if NAME.get()==" " or CITY.get()==" " or ROLLNO.get()==" ":
        lbl_msg.config(text="Please fill all fields!", fg="red")
    else:
        conn = sqlite3.connect("college.db")
        cursor = conn.cursor()
        cursor.execute("UPDATE student SET name=?, city=? WHERE rollno=?",
            (NAME.get(), CITY.get(), ROLLNO.get()))
        conn.commit()
```

```
conn.close()
NAME.set(""); CITY.set(""); ROLLNO.set("")
lbl_msg.config(text="Data Updated!", fg="green")
Read()

def Delete():
    if ROLLNO.get()=="":
        lbl_msg.config(text="Enter RollNo to delete!", fg="red")
    else:
        conn = sqlite3.connect("college.db")
        cursor = conn.cursor()
        cursor.execute("DELETE FROM student WHERE rollno=?", (ROLLNO.get(),))
        conn.commit()
        conn.close()
        NAME.set(""); CITY.set(""); ROLLNO.set("")
        lbl_msg.config(text="Data Deleted!", fg="green")
        Read()

def Exit():
    if messagebox.askyesno("Exit", "Are you sure you want to exit?"):
        root.destroy()

root = Tk()
root.title("Student Registration Form")
root.geometry("700x400")

NAME = StringVar()
CITY = StringVar()
ROLLNO = StringVar()

Label(root, text="Name").grid(row=0, column=0, padx=10, pady=5)
Entry(root, textvariable=NAME).grid(row=0, column=1, padx=10, pady=5)

Label(root, text="City").grid(row=1, column=0, padx=10, pady=5)
Entry(root, textvariable=CITY).grid(row=1, column=1, padx=10, pady=5)

Label(root, text="Roll No").grid(row=2, column=0, padx=10, pady=5)
Entry(root, textvariable=ROLLNO).grid(row=2, column=1, padx=10, pady=5)

lbl_msg = Label(root, text="", fg="red")
lbl_msg.grid(row=3, column=0, columnspan=2)

Button(root, text="Create", command=Create, width=10).grid(row=4, column=0, pady=5)
Button(root, text="Read", command=Read, width=10).grid(row=4, column=1, pady=5)
Button(root, text="Update", command=Update, width=10).grid(row=5, column=0, pady=5)
Button(root, text="Delete", command=Delete, width=10).grid(row=5, column=1, pady=5)
Button(root, text="Exit", command=Exit, width=10).grid(row=6, column=0, columnspan=2, pady=5)

tree = ttk.Treeview(root, columns=("Name", "City", "Rollno"), show="headings")
tree.heading("Name", text="Name")
tree.heading("City", text="City")
tree.heading("Rollno", text="Roll No")
```

```
tree.grid(row=0, column=2, rowspan=7, padx=20, pady=5)
```

```
Database()
```

```
Read()
```

```
root.mainloop()
```

Output:

Name	City	
Vansh	Valsad	39
Nirav	Navsari	13

3. Patient Registration System with SQLite.

```
from tkinter import *
import sqlite3
from tkinter import messagebox

def Database():
    conn = sqlite3.connect("patient.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS patient (
            patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
            firstname TEXT,
            lastname TEXT,
            gender TEXT,
            address TEXT,
            patientno TEXT
        )
    """)
    conn.commit()
    conn.close()

def Create():
    if firstname.get()==" " or lastname.get()==" " or gender.get()==" " or address.get()==" " or patientno.get()==" ":
        messagebox.showerror("Error", "All fields required!")
    else:
        conn = sqlite3.connect("patient.db")
        cursor = conn.cursor()
        cursor.execute("INSERT INTO patient(firstname,lastname,gender,address,patientno) VALUES(?,?,?,?,?)",
            (firstname.get(), lastname.get(), gender.get(), address.get(), patientno.get()))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Record Added")
        Clear()

def Read():
    conn = sqlite3.connect("patient.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM patient")
    rows = cursor.fetchall()
    conn.close()
    listbox.delete(0, END)
    for row in rows:
        listbox.insert(END, row)

def Update():
    if patientno.get()==" ":
        messagebox.showerror("Error", "Enter Patient No to update")
    else:
        conn = sqlite3.connect("patient.db")
        cursor = conn.cursor()
        cursor.execute("UPDATE patient SET firstname=?,lastname=?, gender=?, address=? WHERE patientno=?",
```

```
(firstname.get(), lastname.get(), gender.get(), address.get(), patientno.get()))
conn.commit()
conn.close()
messagebox.showinfo("Success", "Record Updated")
Clear()

def Delete():
    if patientno.get()=="":
        messagebox.showerror("Error", "Enter Patient No to delete")
    else:
        conn = sqlite3.connect("patient.db")
        cursor = conn.cursor()
        cursor.execute("DELETE FROM patient WHERE patientno=?", (patientno.get(),))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Record Deleted")
        Clear()

def Clear():
    firstname.set("")
    lastname.set("")
    gender.set("")
    address.set("")
    patientno.set("")

root = Tk()
root.title("Patient Registration")
root.geometry("500x500")

firstname = StringVar()
lastname = StringVar()
gender = StringVar()
address = StringVar()
patientno = StringVar()

Label(root, text="First Name").pack()
Entry(root, textvariable=firstname).pack()

Label(root, text="Last Name").pack()
Entry(root, textvariable=lastname).pack()

Label(root, text="Gender").pack()
Entry(root, textvariable=gender).pack()

Label(root, text="Address").pack()
Entry(root, textvariable=address).pack()

Label(root, text="Patient No").pack()
Entry(root, textvariable=patientno).pack()

Button(root, text="Create", command=Create).pack(pady=5)
Button(root, text="Read", command=Read).pack(pady=5)
```

```
Button(root, text="Update", command=Update).pack(pady=5)
Button(root, text="Delete", command=Delete).pack(pady=5)
Button(root, text="Clear", command=Clear).pack(pady=5)
```

```
listbox = Listbox(root, width=60, height=10)
listbox.pack(pady=10)
```

```
Database()
root.mainloop()
```

Output:

Patient Registration

First Name

Last Name

Gender

Address

Patient No

Create
Read
Update
Delete
Clear

1 Vansh Prajapati Male Valsad 1
2 Nirav Niyaran Male Navsari 2

4. Member Registration System with Login Details.

```

from tkinter import *
import sqlite3
import tkinter.ttk as ttk
import tkinter.messagebox as tkMessageBox

def Database():
    global conn, cursor
    conn = sqlite3.connect("pythontut.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS member (
            mem_id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
            firstname TEXT,
            lastname TEXT,
            gender TEXT,
            address TEXT,
            username TEXT,
            password TEXT
        )
    """)

def Create():
    if (FIRSTNAME.get()==" " or LASTNAME.get()==" " or GENDER.get()==" " or
        ADDRESS.get()==" " or USERNAME.get()==" " or PASSWORD.get()==" "):
        txt_result.config(text="Please complete the required field!", fg="red")
    else:
        Database()
        cursor.execute("INSERT INTO member(firstname,lastname,gender,address,username,password)
VALUES(?,?,?,?,?,?)",
            (FIRSTNAME.get(), LASTNAME.get(), GENDER.get(), ADDRESS.get(), USERNAME.get(), PASSWORD.get()))
        conn.commit()
        Clear()
        cursor.close()
        conn.close()
        txt_result.config(text="Created a data!", fg="green")

def Read():
    tree.delete(*tree.get_children())
    Database()
    cursor.execute("SELECT * FROM member ORDER BY lastname ASC")
    fetch = cursor.fetchall()
    for data in fetch:
        tree.insert('', 'end', values=(data[1], data[2], data[3], data[4], data[5], data[6]))

```

```
cursor.close()
conn.close()
txt_result.config(text="Successfully read the data from database", fg="black")

def Update():
    if PASSWORD.get()=="":
        txt_result.config(text="Please enter Password to update!", fg="red")
    else:
        Database()
        cursor.execute("UPDATE member SET firstname=?,lastname=?, gender=?, address=?, username=? WHERE
password=?",
            (FIRSTNAME.get(), LASTNAME.get(), GENDER.get(), ADDRESS.get(), USERNAME.get(), PASSWORD.get()))
        conn.commit()
        Clear()
        cursor.close()
        conn.close()
        txt_result.config(text="Updated a data!", fg="green")

def Delete():
    if FIRSTNAME.get()==" or ADDRESS.get()=="":
        txt_result.config(text="Please complete the required field!", fg="red")
    else:
        Database()
        cursor.execute("DELETE FROM member WHERE firstname=? AND address=?", (FIRSTNAME.get(),
ADDRESS.get()))
        conn.commit()
        Clear()
        cursor.close()
        conn.close()
        txt_result.config(text="Deleted a data!", fg="green")

def Clear():
    FIRSTNAME.set("")
    LASTNAME.set("")
    GENDER.set("")
    ADDRESS.set("")
    USERNAME.set("")
    PASSWORD.set("")

def Exit():
    result = tkMessageBox.askquestion("Python: Simple CRUD Application",
        "Are you sure you want to exit?", icon="warning")
    if result == "yes":
        root.destroy()
        exit()
```

```
root = Tk()
root.title("Python: Simple CRUD Application")
root.geometry("900x500")
root.resizable(0, 0)

FIRSTNAME = StringVar()
LASTNAME = StringVar()
GENDER = StringVar()
ADDRESS = StringVar()
USERNAME = StringVar()
PASSWORD = StringVar()

txt_title = Label(root, width=900, font=("arial", 24), text="Python: Simple CRUD Application")
txt_title.pack(side=TOP)

form_frame = Frame(root, bd=8, relief="raise")
form_frame.place(x=10, y=50)

Label(form_frame, text="Firstname:", font=("arial", 16), bd=15).grid(row=0, column=0, sticky="e")
Entry(form_frame, textvariable=FIRSTNAME, width=30).grid(row=0, column=1)

Label(form_frame, text="Lastname:", font=("arial", 16), bd=15).grid(row=1, column=0, sticky="e")
Entry(form_frame, textvariable=LASTNAME, width=30).grid(row=1, column=1)

Label(form_frame, text="Gender:", font=("arial", 16), bd=15).grid(row=2, column=0, sticky="e")
Radiobutton(form_frame, text="Male", variable=GENDER, value="Male", font=("arial", 12)).grid(row=2, column=1, sticky="w")
Radiobutton(form_frame, text="Female", variable=GENDER, value="Female", font=("arial", 12)).grid(row=2, column=1)

Label(form_frame, text="Address:", font=("arial", 16), bd=15).grid(row=3, column=0, sticky="e")
Entry(form_frame, textvariable=ADDRESS, width=30).grid(row=3, column=1)

Label(form_frame, text="Username:", font=("arial", 16), bd=15).grid(row=4, column=0, sticky="e")
Entry(form_frame, textvariable=USERNAME, width=30).grid(row=4, column=1)

Label(form_frame, text="Password:", font=("arial", 16), bd=15).grid(row=5, column=0, sticky="e")
Entry(form_frame, textvariable=PASSWORD, show="*", width=30).grid(row=5, column=1)

btn_frame = Frame(root, bd=8, relief="raise")
btn_frame.place(x=10, y=400)

txt_result = Label(btn_frame)
txt_result.pack(side=TOP)
```

```
Button(btn_frame, width=10, text="Create", command=Create).pack(side=LEFT, padx=5)
Button(btn_frame, width=10, text="Read", command=Read).pack(side=LEFT, padx=5)
Button(btn_frame, width=10, text="Update", command=Update).pack(side=LEFT, padx=5)
Button(btn_frame, width=10, text="Delete", command=Delete).pack(side=LEFT, padx=5)
Button(btn_frame, width=10, text="Clear", command=Clear).pack(side=LEFT, padx=5)
Button(btn_frame, width=10, text="Exit", command=Exit).pack(side=LEFT, padx=5)

tree_frame = Frame(root, bd=8, relief="raise")
tree_frame.place(x=320, y=50)

scrollbary = Scrollbar(tree_frame, orient=VERTICAL)
scrollbarx = Scrollbar(tree_frame, orient=HORIZONTAL)
tree = ttk.Treeview(tree_frame,
                    columns=("Firstname", "Lastname", "Gender", "Address", "Username", "Password"),
                    selectmode="extended", height=15,
                    yscrollcommand=scrollbary.set, xscrollcommand=scrollbarx.set)
scrollbary.config(command=tree.yview)
scrollbarx.config(command=tree.xview)
scrollbary.pack(side=RIGHT, fill=Y)
scrollbarx.pack(side=BOTTOM, fill=X)

tree.heading("Firstname", text="Firstname", anchor=W)
tree.heading("Lastname", text="Lastname", anchor=W)
tree.heading("Gender", text="Gender", anchor=W)
tree.heading("Address", text="Address", anchor=W)
tree.heading("Username", text="Username", anchor=W)
tree.heading("Password", text="Password", anchor=W)

tree.column("#0", stretch=NO, minwidth=0, width=0)
tree.column("#1", stretch=NO, minwidth=0, width=80)
tree.column("#2", stretch=NO, minwidth=0, width=120)
tree.column("#3", stretch=NO, minwidth=0, width=80)
tree.column("#4", stretch=NO, minwidth=0, width=150)
tree.column("#5", stretch=NO, minwidth=0, width=120)
tree.column("#6", stretch=NO, minwidth=0, width=120)
tree.pack()

root.mainloop()
```

Output:

Python: Simple CRUD Application

Firstname:

Lastname:

Gender: ☒ Male ☐ Female

Address:

Username:

Password:

Firstname	Lastname	Gender	Address	Username	Password
Vansh	Prajapati	Male	Valsad	Vansh1503	V@

Successfully read the data from database

Create Read Update Delete Clear Exit